

Modern LLM Architecture

A Visual Guide to Attention, Memory, Sparsity, and Hybrid Models – Three-Chapter Review Pilot

ML/LLM Study Workbooks

Table of contents

1	How to use this workbook	2
2	Notation quick reference	2
3	Chapter 1: Transformer Refresher	3
3.1	Learning objectives	3
3.2	Why this matters	4
3.3	Mental model or analogy	4
3.4	Visual overview	5
3.5	Technical core	5
3.6	Worked example	8
3.7	Compare and contrast	10
3.8	Check your understanding	10
3.9	Applied exercise	11
3.10	Interview lens	11
3.11	Chapter recap	11
3.12	Sources and further reading	12
4	Chapter 2: Anatomy of a Modern Decoder	13
4.1	Learning objectives	13
4.2	Why this matters	13
4.3	Mental model or analogy	13
4.4	Visual overview	14
4.5	Technical core	14
4.6	Worked example	19
4.7	Compare and contrast	20
4.8	Check your understanding	20
4.9	Applied exercise	20
4.10	Interview lens	21
4.11	Chapter recap	21
4.12	Sources and further reading	21
5	Chapter 3: Attention Head Structure and Cache-Efficient Variants	22
5.1	Learning objectives	22
5.2	Why this matters	22
5.3	Mental model or analogy	22
5.4	Visual overview	23
5.5	Technical core	23

5.6	Worked example	25
5.7	Compare and contrast	26
5.8	Check your understanding	27
5.9	Applied exercise	27
5.10	Interview lens	27
5.11	Chapter recap	28
5.12	Sources and further reading	28
6	Answer Key: Chapter 1	29
7	Answer Key: Chapter 2	30
8	Answer Key: Chapter 3	32
9	Build and Version Note	34
9.1	Provenance	34
	Bibliography	34

1 How to use this workbook

Each chapter follows the same structure: learning objectives, why the topic matters, a mental model, a visual overview, the technical core (equations with every symbol defined), a fully worked numeric example, a comparison table, comprehension questions, a common misconception, an applied exercise, an interview-style question, a chapter recap, and sources. Short questions appear near the concept they test, with space left for written answers if you're using this as a workbook rather than only a reading; full solutions are collected in the answer key (Section 6, Section 7, Section 8) rather than revealed inline, so you can attempt each question before checking it. A notation quick reference (Section 2) covers every symbol Chapters 1-3 use.

This is a three-chapter pilot, not the complete workbook

This document contains **only Chapters 1, 2, and 3** of *Modern LLM Architecture: A Visual Guide to Attention, Memory, Sparsity, and Hybrid Models* (workbook 04 of the ML/LLM Study Workbooks series). Chapters 4-8 – restricting attention's reach, mixture-of-experts models, case studies, architecture -> systems consequences, and a synthesis/practice chapter – are planned but **not drafted**. Nothing in this document should be read as implying those chapters exist yet. Chapter 3 also does not yet cover multi-head latent attention (MLA); that mechanism was deliberately deferred out of chapter 3's drafting pass (see `outline.yaml`'s `ch3.drafting_note`) and its eventual chapter placement is not yet decided.

This is a review artifact: it exists to establish the workbook's explanatory depth, visual language, page density, question style, and citation style before the remaining chapters are drafted at scale. Treat any rough edge you find as exactly the kind of thing this pilot is meant to surface early.

2 Notation quick reference

This project maintains a full, growing symbol table across all eight workbooks; later chapters introduce mechanisms this pilot doesn't cover (expert counts and others start in Chapter 5 onward). The subset below is everything Chapters 1-3 actually use – flip back here if a symbol in an equation isn't immediately obvious.

Table 1: Notation used in Chapters 1-3 only; later chapters extend this table

Symbol	Name	Meaning
B	batch size	number of independent sequences processed together
S	sequence length	number of tokens retained in the KV-cache
T_q	query positions (current step)	number of new positions a step computes queries for ($T_q = S$ during prefill, $T_q = 1$ for an ordinary decode step)
V	vocabulary size	number of distinct token ids; also the output-head's logit width
d_{model}	hidden dimension	width of the residual stream, constant across every block
H_q	query heads	number of attention query heads
H_{kv}	key/value heads	number of attention key/value heads ($H_{kv} = H_q$ for MHA, $H_{kv} = 1$ for MQA, $1 < H_{kv} < H_q$ for GQA)
d_{head}	head dimension	width of one attention head, d_{model}/H_q
L	number of layers	count of decoder blocks stacked
d_{ff}	feed-forward expansion dimension	width of a feed-forward sublayer's hidden projection
bytes_per_elem	bytes per cached element	storage width of one stored number (e.g. 2 for bf16); stated explicitly per calculation, never assumed

3 Chapter 1: Transformer Refresher

From Tokens to the Next-Token Distribution

3.1 Learning objectives

By the end of this chapter you should be able to:

- State the decoder-only next-token objective and explain why it is what lets a model generate text one piece at a time.
- Trace a single token's vector as it moves through the residual stream, the attention sublayer, and the feed-forward sublayer of one decoder block.

- Explain causal masking precisely – which positions a given position may and may not attend to – rather than the imprecise “only the previous token” version.
- Distinguish prefill from a single autoregressive decode step, and explain concretely what state gets reused between decode steps.

3.2 Why this matters

Every mechanism in the rest of this workbook – attention variants, positional schemes, sparsity, mixture-of-experts routing – is a modification to the same handful of moving parts introduced here: a residual stream, an attention sublayer, a feed-forward sublayer, and a causal mask. If any of those four feel shaky, later chapters will read as a list of unrelated tricks rather than variations on one theme. This chapter’s job is narrow on purpose: get the shared vocabulary and the prefill/decode distinction solid, so that “GQA changes how many KV heads exist” (Chapter 3) or “MoE changes which parameters activate” (Chapter 5) land as small, precise edits to a picture you already have, not new pictures from scratch.

3.3 Mental model or analogy

Think of the residual stream as a running total printed on a receipt. Each sublayer in the model doesn’t erase what’s already printed – it adds one more line of adjustment underneath the current total. By the time the receipt reaches the bottom of the block, the final total already reflects every earlier adjustment, because each one was added in, in order.

Analogy

The residual stream is a running total on a receipt. Each sublayer reads the current total, computes an adjustment, and prints a new line that *adds* to the total – it never crosses out or overwrites a previous line. Where this breaks down: a paper receipt keeps each line individually readable, so you can always look back and see exactly what line 3 contributed. The residual stream cannot do that. Every adjustment is summed into the *same* fixed-width vector, so by the end you have one number (well, one vector) that reflects every contribution – but you cannot read back which sublayer contributed what just by looking at the final total. That specific limitation is the working assumption behind why interpretability researchers need extra tools (activation patching, probing) to recover what an individual sublayer contributed – the sum alone doesn’t show its work.

3.4 Visual overview

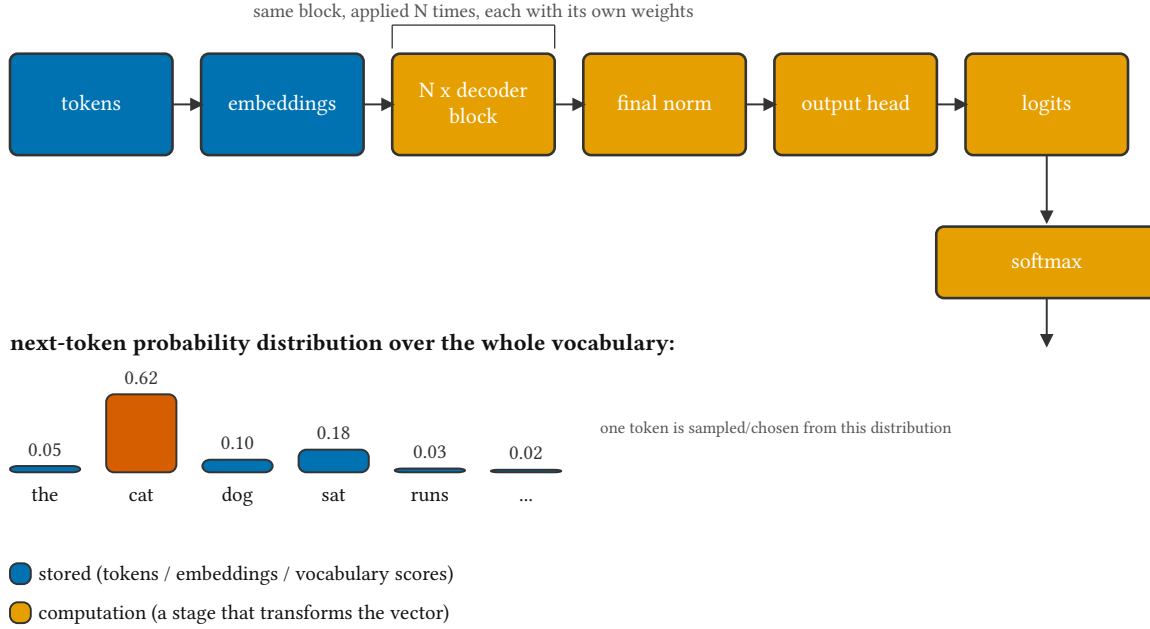


Figure 1: Tokens become vectors, the same block repeats N times with its own weights at each repetition, and only the final softmax turns a vector back into a probability distribution over the vocabulary – nothing “decides” the next token before that last step.

3.5 Technical core

Tokens and embeddings. A tokenizer splits text into a sequence of S discrete tokens, each an integer id into a fixed vocabulary of size V . An embedding table $E \in \mathbb{R}^{V \times d_{\text{model}}}$ maps each token id to a dense vector of width d_{model} (the **hidden dimension**, the width of the residual stream everywhere in the model). For a batch of B sequences of length S , the embedded input has shape (B, S, d_{model}) . We will mostly drop the batch dimension B below and reason about one sequence at a time – everything below happens identically and independently across the batch dimension.

$$x_i = E[\text{token_id}_i], \quad x_i \in \mathbb{R}^{d_{\text{model}}} \quad (1)$$

In words: look up a row of a table. That row is where the model’s only “knowledge” of which token this is enters the computation – from here on, everything operates on vectors, not token identities.

The residual stream. Stack the S embedded vectors into $x \in \mathbb{R}^{S \times d_{\text{model}}}$. This is the residual stream at the input to block 1. Every block updates it the same way:

$$x^{(l+1)} = x^{(l)} + f(\text{Norm}(x^{(l)})) \quad (2)$$

where f is either the attention sublayer or the feed-forward sublayer, and Norm is a normalization step (Chapter 2 covers exactly which normalization and exactly where it sits; for now, treat it as “rescale the vector before reading it”). *In words:* a sublayer never overwrites $x^{(l)}$ – it reads a normalized copy, computes an update, and the update is *added back* onto the original, un-normalized $x^{(l)}$. Shape-wise, $f(\cdot)$ must return something of shape (S, d_{model}) , the same shape it was handed, or the addition would not type-check. That shape constraint is exactly why d_{model} stays constant across every block in a model.

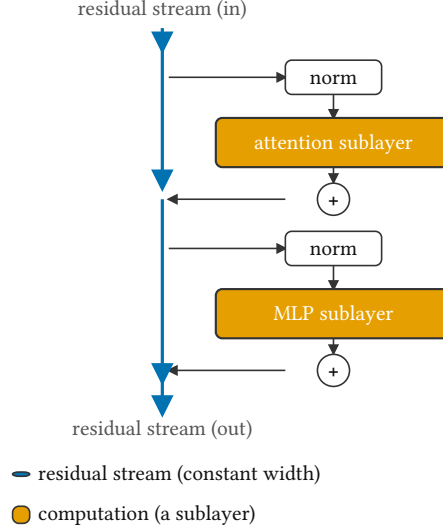


Figure 2: One block, showing the residual stream as the vertical bar and the two sublayers branching off it. Notice the stream’s width never changes – only what gets added to it does.

The attention sublayer. Each block’s attention sublayer projects the normalized stream into three role-specific views:

$$Q = \tilde{x}W_Q, \quad K = \tilde{x}W_K, \quad V = \tilde{x}W_V \quad (3)$$

where $\tilde{x} = \text{Norm}(x)$, and $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ are learned weight matrices. Q, K, V each have shape (S, d_{model}) . Rather than using the full width at once, attention splits d_{model} into H_q **heads** of width $d_{\text{head}} = d_{\text{model}}/H_q$, so each of Q, K, V is reshaped to $(H_q, S, d_{\text{head}})$ and every head runs the attention computation below independently, in parallel, on its own d_{head} -wide slice. *In words:* Q asks “what is this position looking for,” K answers “what does this position offer,” and V is “what this position actually contributes if attended to” – three different linear views of the same vector, learned separately.

Notation note

We write H_q , not just H , for the number of heads – specifically the number of *query* heads. In this chapter every head has its own matching key/value head, so there is no need yet to distinguish a separate key/value head count. Chapter 3 introduces designs (grouped- query and multi-query attention) where the query head count and the key/value head count are genuinely different numbers, at which point the “q” in H_q starts pulling its weight notationally.

For one head, scaled dot-product attention is:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_{\text{head}}}} + M \right) V \quad (4)$$

$QK^\top$ has shape (S, S) – one raw compatibility score per query/key pair. Dividing by $\sqrt{d_{\text{head}}}$ keeps those scores from growing too large as d_{head} grows (large scores would push softmax toward a near-one-hot output before the model has learned anything useful). M is the **causal mask**, and it is the entire reason a decoder can be trained on whole sequences at once without cheating:

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (5)$$

In words: position i 's query is allowed to see keys from position i itself and every position before it ($j \leq i$); a key from any later position ($j > i$) gets an additive $-\infty$, which softmax then sends to exactly zero weight. This is not “attend only to the previous token” – it is “attend to every earlier token, plus yourself, and nothing later.” Figure 3 below makes the difference concrete: notice that the shape of the allowed region is a solid triangle, not a single diagonal line.

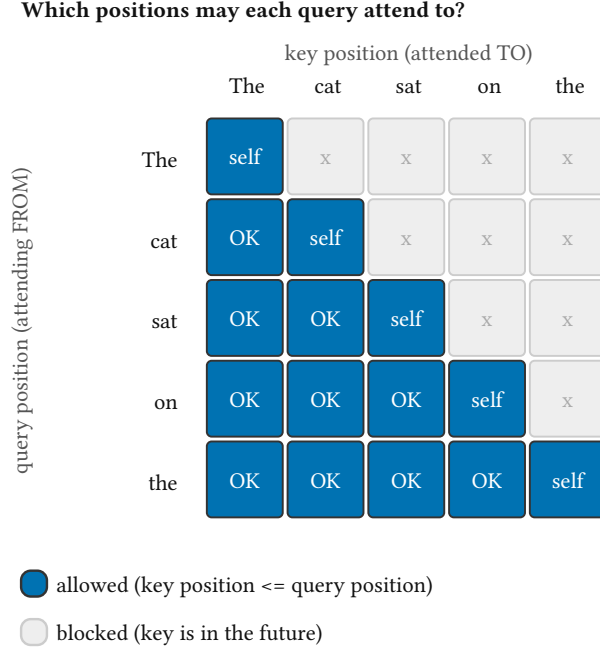


Figure 3: Causal visibility for the chapter’s running example: the allowed region (blue) is everything at or before a row’s own position, not a single step back – position 5 (“the”) can see all 4 prompt tokens plus itself.

After softmax, each row of the (S, S) weight matrix sums to 1 and is used to take a weighted average of V ’s rows, producing a (S, d_{head}) output per head. The H_q heads’ outputs are concatenated back to (S, d_{model}) and passed through one more learned projection W_O before being added back onto the residual stream, per Equation 2.

The feed-forward sublayer. The other sublayer, f in Equation 2, is a position-wise feed-forward network: a small two-matrix (or, as Chapter 2 will show, sometimes three-matrix) network applied independently to every position’s vector, with no cross-position mixing at all – all the cross-position mixing in a block happens in the attention sublayer.

Logits and next-token prediction. After the last block, the residual stream at the final position, $x_S^{(L)} \in \mathbb{R}^{d_{\text{model}}}$, is projected by an output head $W_{\text{out}} \in \mathbb{R}^{d_{\text{model}} \times V}$ into a vector of **logits** over the vocabulary, then turned into a probability distribution:

$$p(\text{next token} = v) = \text{softmax}(x_S^{(L)} W_{\text{out}})_v \quad (6)$$

In words: one more linear projection turns “the model’s final understanding of this position” into one raw score per vocabulary entry, and softmax turns those scores into a valid probability distribution the model (or a sampling procedure) can pick a token from. Nothing about this step is special to the *last* position – every position produces its own next-token distribution, but only the last position’s distribution is the one used to actually extend the sequence, since the earlier positions’ “next tokens” are already fixed by the real input.

Working approximation

This chapter presents one decoder block’s attention and feed-forward sublayers as the complete story. A real model stacks L of these blocks, each with its own weights, and Chapter 2 will show that the normalization step glossed over here (Norm in Equation 2) has a specific, consequential form and placement. Treat everything in this chapter as the shared skeleton that later chapters attach real components to – it is correct as far as it goes, but it goes only as far as “here is the skeleton every modern decoder shares.”

3.6 Worked example

We trace a complete, tiny causal decoder over the prompt “The cat sat on” (4 tokens, processed by prefill) followed by one decode step that generates a 5th token, “the”. Every number below is verified programmatically, not hand-derived – see the Build and Version Note for how.

Toy configuration (deliberately tiny; a real model’s dimensions are hundreds to thousands of times larger, but the mechanics are identical): $d_{\text{model}} = 4$, $H_q = 2$ heads of $d_{\text{head}} = 2$ each, one block. Toy (untrained, hand-picked) embeddings:

Token	Embedding
The	(1, 0, 1, 0)
cat	(0, 1, 0, 1)
sat	(1, 1, 0, 0)
on	(0, 0, 1, 1)
the	(1, 0, 0, 1)

Before tracing the numbers, the same distinction shown here holds at any scale: prefill happens once (conceptually), decode steps happen once per generated token, and each decode step only ever adds one new position’s worth of state.

Implementation-dependent detail

Conceptually, prefill evaluates the prompt positions together and constructs their per-layer K/V state, which is the model this chapter uses throughout. A production serving engine may split a long prompt into chunks for scheduling or memory reasons – chunking does not change the logical distinction this chapter draws between prompt processing and one-position-at-a-time decoding; it is an engineering detail about how “together” gets scheduled, not a different mechanism.

Prefill: build the whole KV-cache from the prompt in one pass

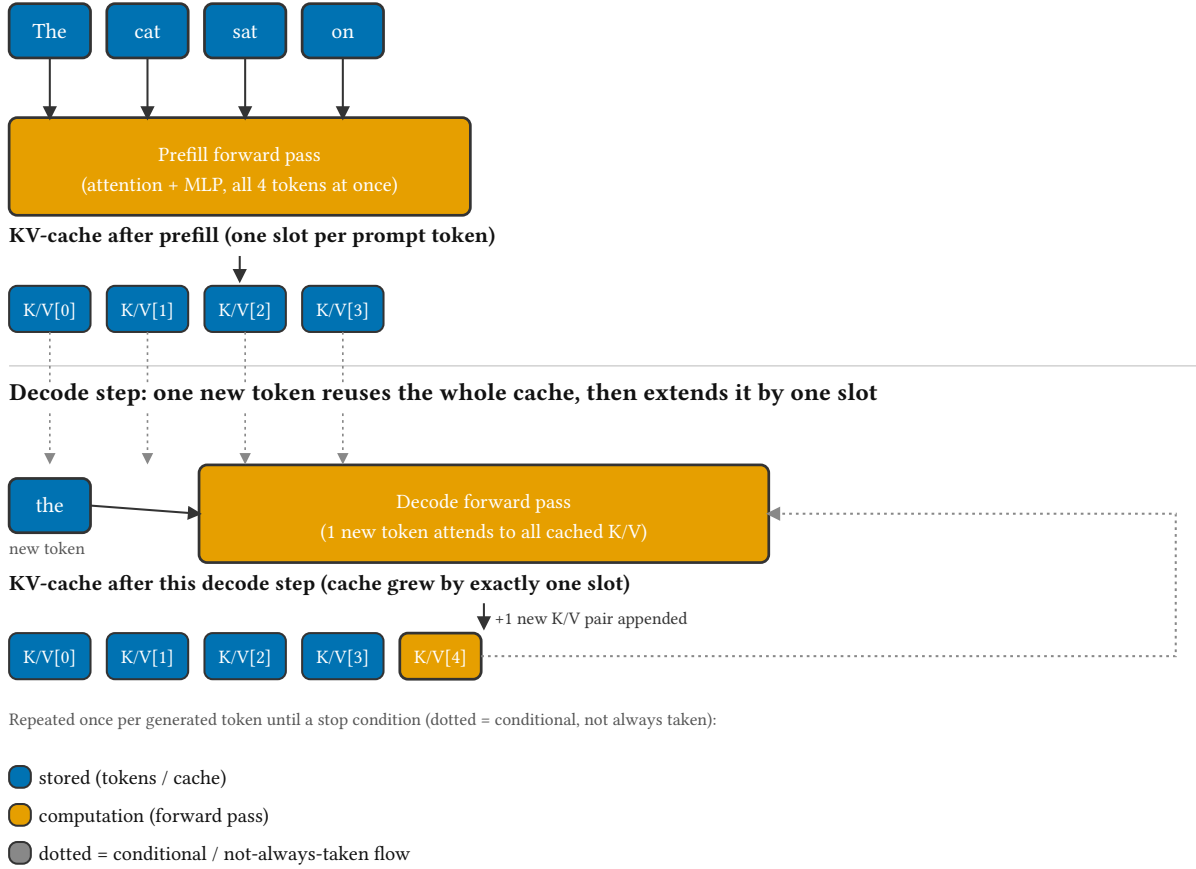


Figure 4: Prefill builds the whole cache from the prompt in one pass; each decode step afterward reads the entire cache but appends exactly one new K/V slot to it. This figure is reused as-is from the project’s existing KV-cache demonstration – the mechanics don’t change between this chapter’s introduction of the idea and Chapter 3’s deeper treatment of it.

Prefill. All 4 prompt tokens are processed together. After computing Q , K , V for all 4 positions and applying the causal mask, head 0’s softmax attention weights are:

Query position	attends to	The	cat	sat	on
The (pos 1)	1.000	–	–	–	–
cat (pos 2)	0.500		0.500	–	–
sat (pos 3)	0.140		0.576	0.284	–
on (pos 4)	0.109		0.449	0.221	0.221

Every row sums to 1.0, and every blocked (future) position is correctly absent, not just small – Equation 5 sent those entries to $-\infty$ before the softmax ever ran. Notice **sat** (row 3) puts most of its weight (0.576) on **cat**, not on itself or on **The**; that specific number is a direct, checkable consequence of the toy W_Q/W_K matrices, not a property of causal masking itself – causal masking only decides *which* positions are eligible, never how the weight is split among them.

Prefill’s output for this block includes the full (4, 4) residual stream after the attention sublayer’s update has been added in, but that residual stream is *not* what gets cached. What actually persists for reuse is each layer’s own K and V projections – computed from that layer’s input, before the attention sublayer’s output is folded back into the residual stream (Chapter 3 gives this retained state a name: the KV-cache). The residual stream is recomputed fresh at every step from whichever tokens are currently being processed; only the per-layer K/V rows persist unchanged across steps. (Some architectures apply a position-dependent transform to K before it is cached – Chapter 3 covers this – so the cached row is not always identical to the raw projection shown here, but it is still derived once per position and reused, never re-derived from the residual stream.)

Decode step. To generate token 5, the model embeds only the new token `the`, computes *its own* Q, K, V (a single new row, not a (5, 4) recomputation), and reuses the 4 cached K/V rows from prefill unchanged. Head 0’s attention weights for this one new query, over all 5 now-available keys, are:

Query position	The	cat	sat	on	the (self)
the (pos 5)	0.075	0.310	0.153	0.153	0.310

Two things to check against Equation 5: this row has *no* blocked entries, because position 5 is the last position and every other position is at or before it; and the weights still sum to 1.0 across all 5 entries. The state reused from prefill (K/V for positions 1 through 4) is bit-for-bit identical to what prefill produced – generating token 5 did not touch positions 1 through 4 at all.

3.7 Compare and contrast

Table 2: Encoder-decoder vs. decoder-only: this workbook’s scope is decoder-only, stated explicitly rather than left implicit

Aspect	Encoder-decoder	Decoder-only
Objective	An encoder reads the full input bidirectionally; a separate decoder generates output conditioned on it	One stack reads and generates using the same causally-masked self-attention throughout
Typical use	Translation, summarization with a distinct source/target	Open-ended text generation, chat, most current general-purpose LLMs
Covered in this workbook	Not covered – named here only for contrast	Every remaining chapter assumes this architecture

3.8 Check your understanding

Full solutions are in the answer key (Section 6), not below, so you can attempt these before checking:

1. **Quick check.** What does the residual stream carry from one block to the next – the raw output of the most recent sublayer, or something else? _____
2. **Explain.** Why does a decoder-only objective let the model generate text one token at a time, in a way that a bidirectional encoder’s objective does not? _____

3. **Calculation.** For a toy config with $d_{\text{model}} = 8$, $H_q = 2$ heads, and $S = 5$: what is d_{head} , and what is the shape of the raw attention score matrix $QK^\top$ for one head, before masking?
4. **Misconception check.** True or false, and why: “causal masking means each token can only attend to the immediately previous token.”
5. **Misconception check.** True or false, and why: “the residual stream is a separate memory database the model writes facts into and reads back later.”

(Space intentionally left for written answers if you are using this as a workbook exercise, not only a reading.)

Common misconception

“The Transformer generates the entire response in one forward pass.” It does not. One forward pass produces exactly one next-token distribution, for one additional position. A 50-token response requires at least 50 forward passes (one prefill pass over the prompt, then one decode step per generated token) – the model has no mechanism for “seeing ahead” to token 50 while generating token 2. This misconception is worth taking seriously because it is exactly backward from how latency actually behaves in production: generation cost scales with *response* length, almost independently of how long the *prompt* was processed in a single prefill pass.

3.9 Applied exercise

Pick a toy config of your own: choose d_{model} , number of layers L , and number of heads H_q (make sure d_{model} is divisible by H_q). For a single token’s forward path through **one** full block, write down the shape of every intermediate tensor: the input vector, $Q/K/V$ before and after splitting into heads, the per-head attention output, the concatenated output, and the vector added back onto the residual stream. Check your shapes against Equation 3 and Equation 4 – every shape should be traceable to d_{model} and H_q alone. If you want a check with real numbers instead of just shapes, redo the trace above with your own d_{model}/H_q and toy embeddings; the same programmatic verification approach applies.

3.10 Interview lens

A common interview framing is: “Walk me through what happens differently between processing a 500-token prompt for the first time versus generating the 501st token.” A strong answer distinguishes prefill (conceptually, all 500 positions evaluated together, building per-layer K/V state for the whole prompt – possibly split into chunks by the serving engine, which doesn’t change the logical picture) from a decode step (one new position, reusing the 500 cached K/V rows unchanged, only computing K/V for the new token) – and explicitly rejects “the model re-reads the whole prompt every time,” which is the same misconception as this chapter’s headline one, just phrased in systems terms. See workbook 02 (ML Interview Practice) for more on how to structure this kind of answer under time pressure.

3.11 Chapter recap

- A decoder-only model conditions strictly on earlier positions; nothing about its objective lets it “see ahead.”
- The residual stream is additive and constant-width: sublayers add updates to it, they never replace it or change its dimensionality.
- Attention has three roles (Q , K , V), a causal mask that allows every earlier-or-equal position (not just the immediately previous one), and a softmax that turns raw scores into a weighted average.

- Prefill conceptually evaluates the whole prompt together and produces cached per-layer K/V state (the residual stream itself is not that cached state); each decode step reuses that cached state and adds exactly one new position's worth of new state.
- Generating N tokens costs at least N forward passes – there is no single-pass shortcut.

3.12 Sources and further reading

Primary source for this chapter: [1], the original Transformer paper, which this chapter's notation and equations follow directly. Secondary/supporting: [2], used here only for the encoder-decoder vs. decoder-only framing convention, not for any equation.

4 Chapter 2: Anatomy of a Modern Decoder

The Components Behind Today's LLMs

4.1 Learning objectives

By the end of this chapter you should be able to:

- Explain RMSNorm's formula and state precisely what it drops relative to LayerNorm.
- Explain RoPE's rotation well enough to say why it encodes *relative* rather than absolute position.
- Explain why a gated feed-forward layer needs three weight matrices instead of two, and what that costs in parameters.
- For any one of RMSNorm, pre-norm placement, RoPE, SwiGLU, or tied embeddings, say whether it is an architectural convention (a common but non-forced choice) or something closer to a fundamental requirement – and justify the answer rather than asserting it.

4.2 Why this matters

Chapter 1 gave you the skeleton every decoder shares: a residual stream, an attention sublayer, a feed-forward sublayer, a causal mask. If you opened a 2017 Transformer and a 2025-era open-weight model side by side, the skeleton would look identical – and almost every detail *inside* it would be different. This chapter fills in what actually changed, component by component, and – just as importantly – asks of each one whether it's load-bearing or just a common convention. That distinction matters practically: a convention is something you could swap out without breaking the model's ability to function; a requirement is something the architecture depends on. Confusing the two is a fast way to either over-explain a trivial implementation choice or under-explain something that quietly determines a model's memory footprint.

4.3 Mental model or analogy

This chapter reuses Chapter 1's residual-stream picture rather than introducing a second, competing analogy – the “running total on a receipt” from Chapter 1 still describes exactly what's happening here. What's new in this chapter is *which specific operation* fills each slot in that picture (Figure 5 below), not a new mental model for the stream itself.

4.4 Visual overview

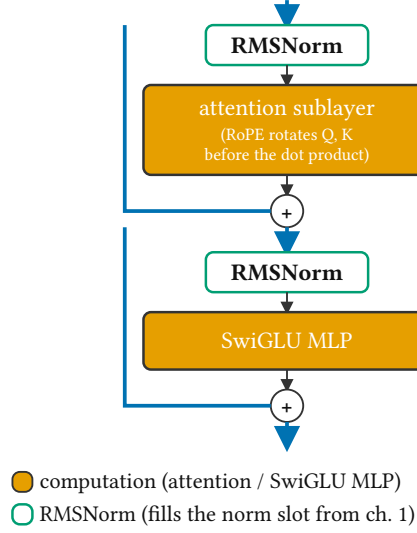


Figure 5: The same residual-stream skeleton from Chapter 1, now filled in with the specific components a modern decoder actually uses: RMSNorm in the norm slots, RoPE applied inside attention (not as a separate block), and a SwiGLU-gated MLP in the feed-forward slot.

4.5 Technical core

RMSNorm. Chapter 1 left **Norm** in Equation 2 unspecified. The original Transformer used LayerNorm, which both re-centers (subtracts the mean) and re-scales (divides by the standard deviation) a vector, then applies a learned scale and a learned bias. RMSNorm drops both the mean-subtraction *and* the learned bias, keeping only a rescale by root- mean-square magnitude:

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}_\epsilon(x)} \odot \gamma, \quad \text{RMS}_\epsilon(x) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{k=1}^{d_{\text{model}}} x_k^2 + \epsilon} \quad (7)$$

where $x \in \mathbb{R}^{d_{\text{model}}}$ is one position's vector, $\gamma \in \mathbb{R}^{d_{\text{model}}}$ is a learned per-dimension scale, $\odot$ is elementwise multiplication, and ϵ is a small fixed constant (not learned) added purely to keep the denominator away from zero when a vector's RMS magnitude is near zero – it has no other role in the formula. *In words:* measure the vector's overall magnitude (RMS), divide it out so the vector has unit RMS, then let a learned per-dimension scale stretch or shrink each dimension back. There is no learned shift and no mean-subtraction step at all.

Common misconception

“RMSNorm and LayerNorm are identical except for naming.” They are not. RMSNorm drops mean-centering (the $x - \text{mean}(x)$ step) *and* the learned bias term, not just the bias. Dropping mean-centering is the more consequential change: it means RMSNorm never asks “is this vector’s average component large or small,” only “is this vector’s overall magnitude large or small.” Both choices are motivated by the same practical benefit – fewer operations per normalization call, at a cost most current large decoders have judged acceptable – but “fewer operations” and “identical except for naming” are very different claims.

Where normalization sits. Equation 2 wrote **Norm** as happening *before* handing the vector to a sublayer – that placement is called **pre-norm**. The original Transformer instead normalized *after* the sublayer’s output was added back onto the stream (**post-norm**). A third option, **sandwich-norm**, applies normalization in both places around the same sublayer.

Table 3: Pre-norm vs. post-norm vs. sandwich-norm: what changes is placement, not the normalization formula itself

Placement	Where norm sits	Typical motivation
Pre-norm	Before the sublayer reads the stream	Training stability at large depth: gradients flow through an unnormalized residual path all the way back
Post-norm	After the sublayer’s output is added to the stream	The original placement; keeps the stream itself bounded, at some training-stability cost as depth grows
Sandwich-norm	Both before and after the sub-layer	Attempts to recover some of post-norm’s boundedness without losing pre-norm’s stability

Same sublayer, same residual add -- only where "norm" sits changes

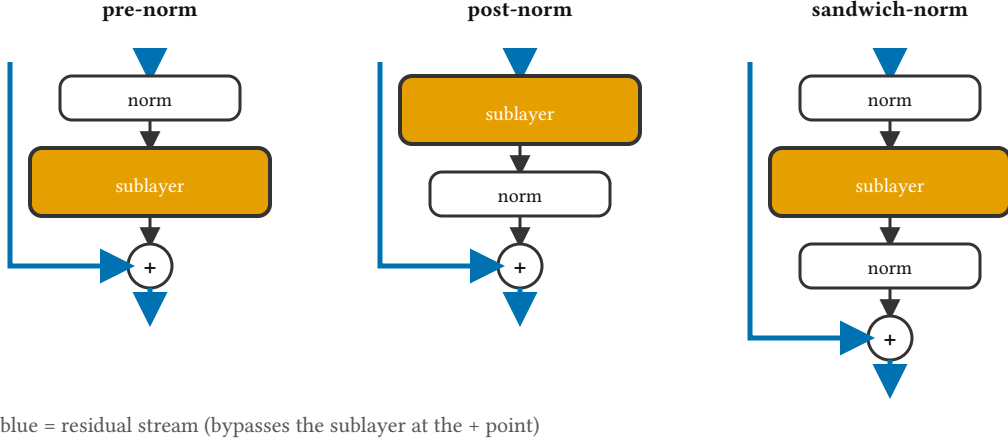


Figure 6: The same sublayer and the same residual addition in all three panels – only where the norm box sits changes.

Working approximation

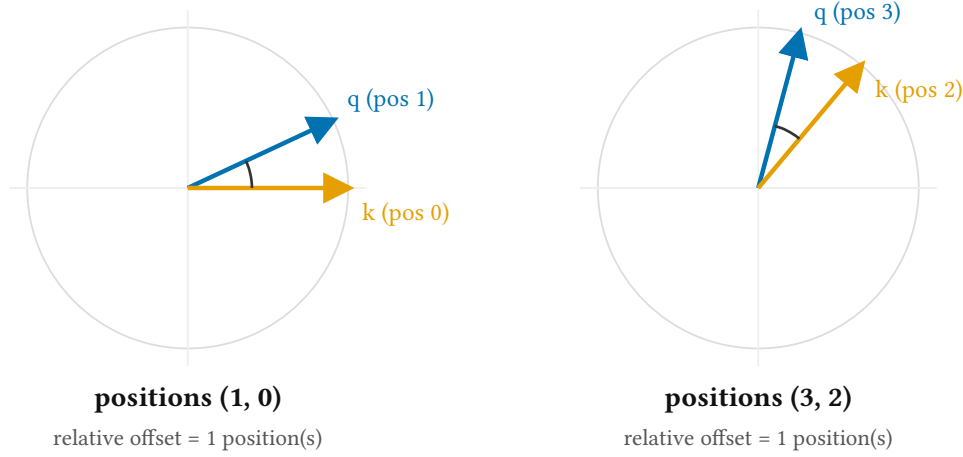
“Pre-norm is more stable to train at depth” is a widely-repeated, well-supported rule of thumb, not a proof that pre-norm dominates post-norm on every axis. Pre-norm’s stability comes with a representational-capacity cost as depth grows, which is part of why sandwich-norm exists at all – if pre-norm were unconditionally better, there would be no reason to pay for a second normalization step. Treat “which placement is best” as a genuine, config-dependent tradeoff, not a settled question.

RoPE. Positional information has to enter the model somewhere, since attention itself (Equation 4) treats its inputs as an unordered set – nothing in $QK^\top$ depends on token order unless something makes it depend on order. Rotary position embeddings (RoPE) inject position by rotating each query and key vector by an angle proportional to its position, *before* the dot product in Equation 4:

$$q'_m = R_m q_m, \quad k'_n = R_n k_n \quad (8)$$

where $q_m, k_n \in \mathbb{R}^{d_{\text{head}}}$ are a query/key pair at positions m and n , and R_m is a rotation matrix built from angles $m\theta_1, m\theta_2, \dots$ that grow with position m (in practice d_{head} is split into 2D pairs, each pair rotated at its own fixed frequency θ_j ; Figure 7 shows one such pair). The reason this matters: rotating *both* vectors by an amount tied to their own absolute position leaves the *angle between them* – and therefore their dot product $q'_m{}^\top k'_n$ – dependent only on the relative offset $m - n$, not on m and n individually.

Same relative offset, different absolute positions -> same angle between q and k



Both panels: the angle traced between q and k is identical, because each vector is rotated by its OWN position, so only the difference in position survives.

Simplified model: real RoPE rotates several 2D subspaces at different frequencies at once; this figure shows one subspace at one frequency to keep the geometry checkable by eye.

Figure 7: Two different absolute-position pairs, (1,0) and (3,2), both with a relative offset of 1 – notice the angle traced between the rotated q and k vectors is the same in both panels.

Common misconception

“RoPE adds a learned position vector to every token embedding.” It does not add anything to the embedding, and it is not learned. RoPE *rotates* the query and key vectors, inside the attention computation, by a fixed (not learned) angle that depends on position. The embedding table from Chapter 1 (Equation 1) is untouched – RoPE’s effect is purely inside Equation 4, applied fresh at every layer that has attention, not once at the input.

Gated feed-forward networks and SwiGLU. Chapter 1 described the feed-forward sublayer as “a small two-matrix network,” deferred until now. A plain feed-forward layer is:

$$\text{FFN}_{\text{plain}}(x) = \sigma(xW_1)W_2 \quad (9)$$

with $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, and σ an elementwise activation. d_{ff} (the **expansion dimension**) is typically several times wider than d_{model} . A **gated** feed-forward layer, of which SwiGLU is the specific variant most current open decoders use, adds a second, parallel projection that gates the first:

$$\text{SwiGLU}(x) = (\text{Swish}(xW_1) \odot (xW_3)) W_2 \quad (10)$$

with $W_1, W_3 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ – **three** weight matrices where the plain version had two. *In words:* one branch (W_1 , activated) acts as a gate; a second, purely linear branch (W_3) carries a value; the two are multiplied elementwise before the shared output projection W_2 . Nothing here makes the layer

sparse the way a mixture-of-experts layer is (Chapter 5) – every weight in W_1 , W_2 , and W_3 is used for every token; SwiGLU is a different, denser computation per token, not a routing mechanism.

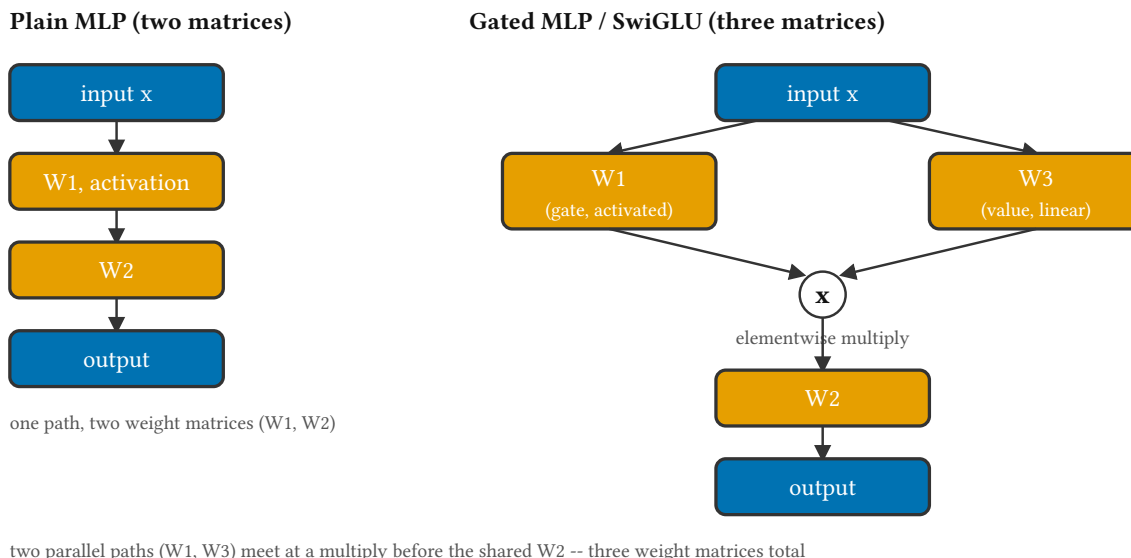


Figure 8: A plain two-matrix MLP (left) against a gated MLP (right): the extra matrix creates a second parallel branch that multiplies against the first, it does not add a third sequential step.

Common misconception

“SwiGLU makes the feed-forward block sparse like an MoE layer.” It does not. Every parameter in a SwiGLU feed-forward layer participates in every token’s forward pass, exactly like the plain version – the only change is *within* one dense computation (an extra multiplicative branch), not *which* parameters get used. Sparsity, in the sense Chapter 5 will use the word, means a router activates only a subset of available parameters per token. SwiGLU has no router and activates 100% of its own weights every time, same as a plain FFN.

Tied vs. untied embeddings. The input embedding table E from Equation 1 and the output head W_{out} from Equation 6 have transposed shapes ($V \times d_{\text{model}}$ and $d_{\text{model}} \times V$) – some models set $W_{\text{out}} = E^T$ (**tied** embeddings, one shared table doing both jobs), others learn two separate matrices (**untied**). Tying halves that specific parameter cost, at the cost of forcing the same table to serve two different roles (looking a token up vs. scoring how token-like a hidden vector is).

Implementation-dependent detail

Whether a given model ties its embeddings is a genuine implementation choice that varies across model families, and this workbook does not have a primary source specifically dedicated to *when* tying is preferred – a tracked sourcing gap (see the Build and Version Note). Treat “tied embeddings” as a real, checkable fact about any specific model’s config – not as a claim this workbook makes a historical or performance argument about.

Convention vs. requirement, applied. Of everything in this chapter, RoPE-vs-some-other-positional-scheme and tied-vs-untied-embeddings are conventions: a model is a well-defined, functioning decoder either way, and real, current models make both choices differently. Pre-norm-vs-post-norm is closer to a requirement-shaped tradeoff at large depth – it is not free to ignore once L is large, even though small models tolerate either. RMSNorm vs. LayerNorm is closest to a pure convention: it changes cost, not what the model is capable of expressing.

Common misconception

“Every modern decoder uses exactly the same block ordering.” It does not. This chapter’s components (RMSNorm, pre-norm, RoPE, SwiGLU) describe a common, current combination, not a single mandatory recipe. Norm placement alone has three real variants in production models (Table 3); tying is a per-model choice; some models still use post-norm. “Modern decoder” describes a family of closely related designs, not one design.

4.6 Worked example

Parameter cost of gating. Verified programmatically (see the Build and Version Note), with $d_{\text{model}} = 8$ and biases excluded from the count (a stated assumption):

Table 4: Gating adds a third matrix – at the same expansion width, that’s a flat 1.5x parameter cost, which is exactly why gated models typically shrink d_{ff} instead of accepting it

Design	d_{ff}	Formula	Parameters
Plain MLP	32	$2 d_{\text{model}} d_{\text{ff}}$	512
Gated MLP, same d_{ff}	32	$3 d_{\text{model}} d_{\text{ff}}$	768 (1.5x plain)
Gated MLP, compensated d_{ff}	21	$3 d_{\text{model}} d_{\text{ff}}$	504 (0.98x plain)

Holding d_{ff} fixed, gating is exactly 1.5x the plain design’s parameters, because the formula gains one more $d_{\text{model}} \times d_{\text{ff}}$ term (3 matrices instead of 2, all the same size). Rather than accept that 1.5x, many real gated-MLP models shrink d_{ff} (here, to about $\frac{2}{3}$ of the plain design’s width) to land back near the plain design’s total parameter count. Both rows are *implementation choices* – nothing about SwiGLU forces either one – but the 1.5x number is a fixed, checkable consequence of adding a third same-sized matrix, not a rough estimate.

4.7 Compare and contrast

Table 5: Two ways to inject positional information, compared on what they cost and how they behave outside the training distribution

Aspect	Learned absolute positional embedding	RoPE
What it encodes	Each position gets its own learned vector	A rotation angle, applied inside attention, derived from position
Extrapolation to longer sequences than trained on	Poor – positions beyond the trained range have no learned vector at all	Better, though not unlimited – the rotation formula is defined for any position, but very long offsets still stress the model in practice
Parameter cost	One full vector per position, up to a fixed max length	None – RoPE has no learned parameters of its own

4.8 Check your understanding

Full solutions are in the answer key (Section 7):

1. **Calculation.** For a query/key pair at positions 1 and 0 (relative offset 1), and a second pair at positions 3 and 2 (also relative offset 1), what does RoPE guarantee about the two pairs' dot products, and what does it *not* guarantee? _____
2. **Compare.** Name the two operations RMSNorm omits relative to LayerNorm.

3. **Explain.** Why does a gated feed-forward layer need three weight matrices instead of two – what does the third matrix do that the first one doesn't already?

4. **Calculation.** Using the formulas in this chapter, what is the parameter count of a plain MLP with $d_{\text{model}} = 16$, $d_{\text{ff}} = 64$? _____
5. **Misconception check.** Is pre-norm's popularity mainly about training stability or mainly about final model quality? _____
6. **Misconception check.** True or false, and why: "RoPE adds a learned position vector to every token embedding." _____
7. **Misconception check.** True or false, and why: "SwiGLU makes the feed-forward block sparse like an MoE layer." _____

(Space intentionally left for written answers if you are using this as a workbook exercise, not only a reading.)

4.9 Applied exercise

Here is a (fictional, illustrative) model config excerpt:

```
hidden_size: 2048
num_attention_heads: 16
intermediate_size: 5504
hidden_act: "silu"
```

```
rms_norm_eps: 1e-5
tie_word_embeddings: false
rope_theta: 10000.0
```

From this excerpt alone: (a) which normalization scheme does this model use, and how do you know from a field name alone versus what you’d need to check the model’s own code to be sure? (b) does it use a gated feed-forward layer – what field is the giveaway? (c) is `intermediate_size` closer to $4 \times \text{hidden_size}$ (a plain-MLP-style ratio) or closer to $\frac{2}{3} \times 4 \times \text{hidden_size}$ (a gated, compensated ratio, per this chapter’s worked example)? (d) are embeddings tied? Write your reasoning for each, not just the answer – this is the same skill Chapter 6’s case studies will ask of you at real-model scale.

4.10 Interview lens

“Why does this model use RoPE instead of learned positional embeddings?” A strong answer states RoPE’s relative-position property (Equation 8) as the mechanism, names at least one concrete consequence (no learned positional parameters; different, generally better long-context extrapolation behavior than a learned table with a fixed max length), and resists the temptation to just say “it’s more modern” without saying what specifically changed. *“Walk me through what SwiGLU changes about the MLP”* is answered well by describing the extra parallel branch and the elementwise multiply (Equation 10), explicitly including the 1.5x-or-compensated parameter tradeoff from this chapter’s worked example – an answer that skips the parameter cost entirely is incomplete for a systems-flavored interview.

4.11 Chapter recap

- RMSNorm drops both mean-centering and the learned bias LayerNorm has, keeping only a magnitude rescale (plus a small fixed ϵ to avoid dividing by zero).
- Pre-norm, post-norm, and sandwich-norm describe *where* normalization sits relative to a sublayer and the residual add – not three different normalization formulas.
- RoPE rotates queries and keys by a position-dependent angle inside attention, so their dot product depends only on relative offset – it adds nothing to the embedding table and learns nothing.
- A gated (SwiGLU-style) feed-forward layer adds one parallel matrix and one elementwise multiply; at matched expansion width that’s a flat 1.5x parameter cost, which is why real models often shrink the expansion width to compensate.
- Tied vs. untied embeddings and RoPE-vs-alternative are conventions; norm placement is closer to a real training-stability tradeoff at depth. None of this chapter’s components is used identically by every current model.

4.12 Sources and further reading

Primary sources for this chapter: [3] for RoPE, [4] for RMSNorm, [5] for SwiGLU/gated feed-forward layers. Secondary/ supporting: [2], used for the norm-placement comparison framing and to cross-check which real model families use which combination of these components; this source is actively updated, so treat any model-specific detail drawn from it as current as of its last verification (see the Build and Version Note), not timeless. This chapter works around two tracked sourcing gaps: tied embeddings has no dedicated primary source, and the sandwich-norm claim leans on a secondary source rather than a model’s own technical report (both noted in the Build and Version Note).

5 Chapter 3: Attention Head Structure and Cache-Efficient Variants

Why Fewer Key/Value Heads Means a Smaller Cache – Not a Smaller Model

5.1 Learning objectives

By the end of this chapter you should be able to:

- Distinguish query-head count (H_q) from key/value-head count (H_{kv}), and write $Q/K/V$'s shapes before and after splitting into heads.
- Explain how multiple query heads can share a key/value head, and distinguish MHA, MQA, and GQA structurally on that one axis.
- Calculate a decoder's logical KV-cache size under stated assumptions, and state which projection parameters change when H_{kv} changes (and which don't).
- Explain why reducing H_{kv} lowers cache memory and memory traffic without reducing every part of attention computation by the same factor.
- Identify tradeoffs among MHA, MQA, and GQA without claiming any one is universally best, and distinguish an architecture-level cache calculation from measured serving-memory usage.

5.2 Why this matters

Chapters 1-2 gave every block one query head per key/value head, without saying that was a choice. It is. The number of key/value heads a model uses is an independent design knob from the number of query heads, and it is the single knob most responsible for how much memory autoregressive decoding needs per active sequence. Get this chapter's distinction wrong and two very different mistakes follow: overstating what changes (claiming a smaller cache means a proportionally smaller or faster model) or understating it (missing that cache memory, not FLOPs, is usually the actual serving bottleneck this chapter is about).

5.3 Mental model or analogy

Analogy

Queries are readers asking different questions; keys are index entries used to decide where to look; values are the information retrieved at that index entry. Standard attention (MHA) gives every reader their own private index and information store. MQA makes all readers share one shared index and store. GQA gives each *group* of readers a shared index and store – a middle ground between the two.

Where this breaks down: keys and values are learned vector projections, not literal documents. Sharing a key/value head changes how much distinct *representational capacity* the model has to work with, not merely how many copies of a document are stored – two readers sharing an index can no longer each shape that index around only their own question.

5.4 Visual overview

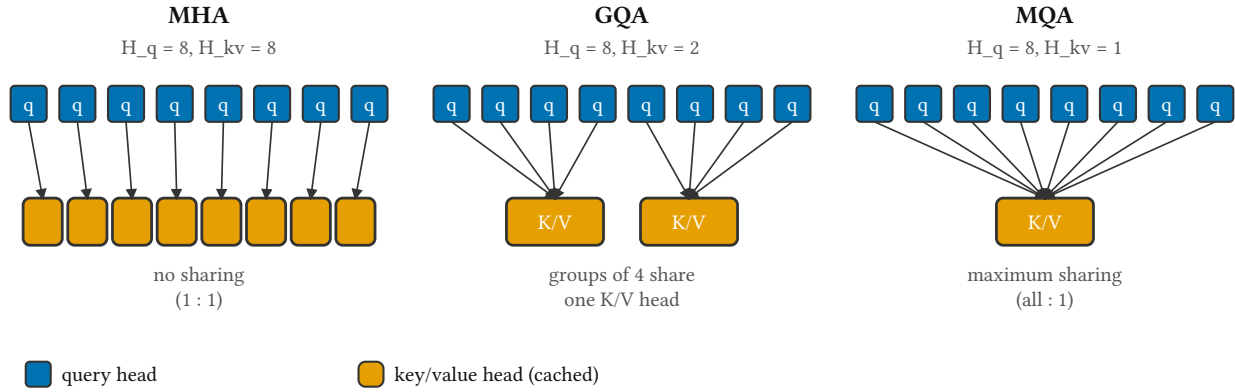


Figure 9: MHA, GQA, and MQA shown with the same $H_q = 8$ query heads in every panel – nothing removes a query head. What changes is how many key/value heads those query heads fan into: 8 (no sharing), 2 (groups of 4 share one), or 1 (all share one). Source: mechanism per [6], [7].

5.5 Technical core

This chapter uses H_q (query heads), H_{kv} (key/value heads), d_{head} (head dimension, with $d_{\text{model}} = H_q d_{\text{head}}$), S (tokens retained in the cache), T_q (query positions processed in the *current* step – $T_q = S$ during prefill, $T_q = 1$ for an ordinary decode step), L (layers), and `bytes_per_elem` (storage width per cached number, e.g. 2 for bf16). Logical shapes, head axis explicit (physical memory layouts vary by implementation):

$$Q : (B, T_q, H_q, d_{\text{head}}), \quad K : (B, S, H_{kv}, d_{\text{head}}), \quad V : (B, S, H_{kv}, d_{\text{head}}) \quad (11)$$

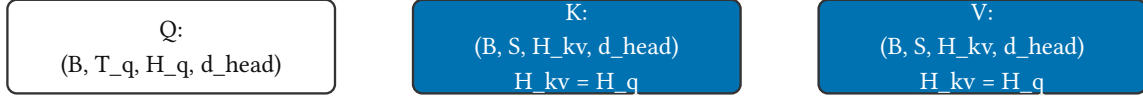
MHA, MQA, GQA: one axis, three points. Standard multi-head attention (MHA, [1]) sets $H_{kv} = H_q$: every query head has its own key/value head. Multi-query attention (MQA, [6]) sets $H_{kv} = 1$: every query head shares the same single key/value head. Grouped-query attention (GQA, [7]) sets $1 < H_{kv} < H_q$: query heads are split into H_q/H_{kv} equal groups (this simple equal-group presentation requires H_{kv} to divide H_q), each group sharing one key/value head. Figure 9 shows all three at $H_q = 8$.

Common misconception

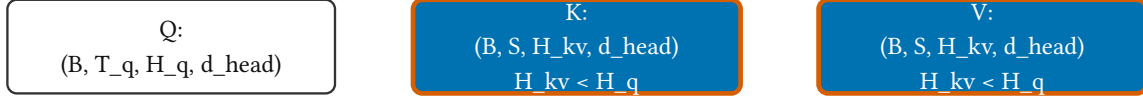
“GQA reduces the number of query heads.” It does not. H_q is untouched by any of these three variants – Figure 9 draws the same 8 query heads in every panel. Only H_{kv} changes.

What stays fixed, what changes. Across MHA/MQA/GQA at the same H_q and d_{head} : the query projection’s shape, the per-head attention *output* shape, the concatenated output width, and the output projection width are all unchanged – every one of them depends on H_q , not H_{kv} . What changes is the K/V projections’ output width and, per position, how many K/V vectors get stored. Figure 10 marks the one shape that differs.

MHA



GQA



Q's shape never changes between MHA and GQA -- only K/V's head axis (outlined in red) shrinks.

Group size = H_q / H_{kv} (e.g. H_q=8, H_{kv}=2 -> each K/V head serves a group of 4 query heads).

Figure 10: Only K/V 's head axis (H_{kv}) shrinks between MHA and GQA; Q 's shape is untouched.

Projection parameters. Ignoring biases, per layer:

$$P_Q = d_{\text{model}} H_q d_{\text{head}}, \quad P_K = d_{\text{model}} H_{kv} d_{\text{head}}, \quad P_V = d_{\text{model}} H_{kv} d_{\text{head}}, \quad P_O = H_q d_{\text{head}} d_{\text{model}} \quad (12)$$

Only P_K and P_V depend on H_{kv} ; P_Q and P_O depend only on H_q . Shrinking H_{kv} shrinks two of these four terms, not all four – the worked example below turns this into a concrete ratio.

KV-cache size. The quantity these mechanisms actually optimize is not parameter count but cache memory:

$$M_{KV} = 2 B S L H_{kv} d_{\text{head}} \text{ bytes_per_elem} \quad (13)$$

where the factor 2 counts keys *and* values. This is a **logical, theoretical minimum** – per AGENTS.md's distinction between logical and measured memory, it explicitly excludes allocator overhead, block/page metadata, fragmentation, temporary attention workspaces, framework buffers, duplicated or padded storage, sharding-specific per-device layouts, prefix-sharing effects, and any cache quantization not already reflected in `bytes_per_elem`.

Common misconception

“The KV-cache formula gives exact observed GPU memory usage.” It does not. Equation 13 is a logical minimum computed from a config; what `nvidia-smi` or a profiler reports also includes every exclusion listed above, and will always be larger.

Decode-time computation. At each decode step, T_q new query positions are computed (ordinarily $T_q = 1$; chunked prefill and speculative-decoding verification – both later topics, named here only as forward references – can make $T_q > 1$). Every one of the H_q query heads still produces its own attention scores; a shared key/value head is simply reused (logically broadcast) across every query head in its group. Reducing H_{kv} principally reduces what gets *stored* and *moved* – it does not reduce the number of query-side dot products, so it is not accurate to say GQA makes attention some fixed multiple faster; the win this chapter is about is memory and memory-bandwidth, not FLOPs.

5.6 Worked example

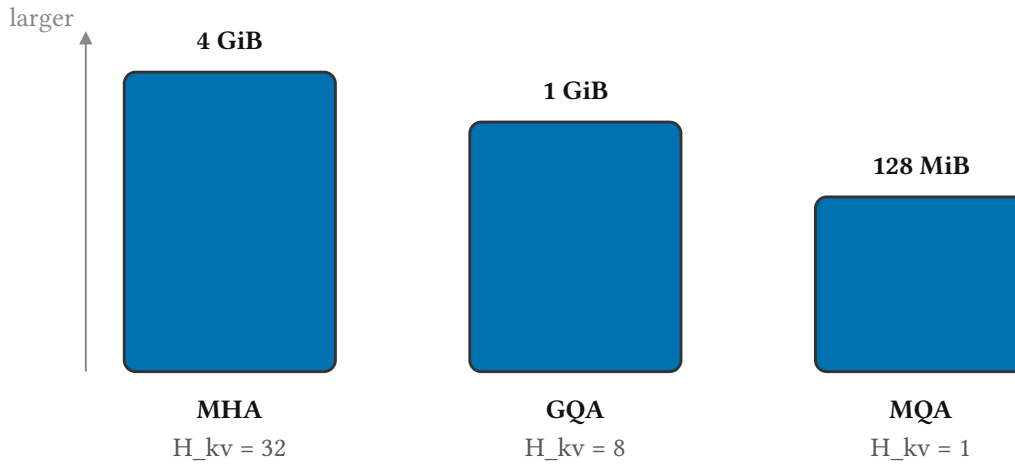
KV-cache memory. Base config: $L = 32$, $H_q = 32$, $d_{\text{head}} = 128$ (so $d_{\text{model}} = 4096$), $S = 8192$, $B = 1$, bf16 (bytes_per_elem = 2) – verified programmatically (see the Build and Version Note):

Table 6: KV-cache size for one sequence at $S = 8192$; the reduction factor equals $H_{kv, \text{variant}} / H_{kv, \text{MHA}}$ exactly

Variant	H_{kv}	Equation 13 result	vs. MHA
MHA	32	4,294,967,296 B = 4 GiB	1x
GQA	8	1,073,741,824 B = 1 GiB	0.25x (4x smaller)
MQA	1	134,217,728 B = 128 MiB	0.03125x (32x smaller)

Logical KV-cache size per sequence (L=32, H_q=32, d_head=128, S=8192, B=1, bf16)

bar height is log2-scaled for visibility -- printed values are the actual linear byte counts



Logical minimum only -- excludes allocator overhead, page metadata, fragmentation, attention workspaces, and framework buffers. Not a measured GPU-memory number.

Figure 11: The same three numbers as Table 6 – bar height is log2-scaled so the smallest bar stays visible; the printed values are the actual linear byte counts.

Cache size scales *linearly* in each of B , S , L , H_{kv} , and bytes_per_elem – e.g. at $B = 16$ active sequences instead of 1, every number in Table 6 simply multiplies by 16 (4 GiB $\rightarrow$ 64 GiB for MHA, 1 GiB $\rightarrow$ 16 GiB for GQA, 128 MiB $\rightarrow$ 2 GiB for MQA); nothing about the formula’s *shape* changes.

Projection parameters, same config. Per layer, from Equation 12:

Table 7: Per-layer attention parameters for the same base config – $K+V$ shrinks by exactly the cache-reduction ratio, but the complete attention block (which also includes Q and O , both fixed) shrinks by much less

Variant	P_Q	P_K+P_V	P_O	Total ($Q+K+V+O$)	$K+V$ vs. MHA	Total vs. MHA
MHA	16,777,216	33,554,432	16,777,216	67,108,864	1x	1x
GQA	16,777,216	8,388,608	16,777,216	41,943,040	0.25x	0.625x
MQA	16,777,216	1,048,576	16,777,216	34,603,008	0.03125x	0.516x

GQA’s $K+V$ parameters drop by exactly 4x, matching its cache reduction – but the *complete* attention block (adding the unchanged P_Q and P_O) drops by only 1.6x (to 0.625 of MHA’s total), and MQA’s 32x $K+V$ reduction becomes less than a 2x reduction (0.516x) once Q and O are included. This is Equation 12’s point made concrete: K/V savings do not translate one-for-one into whole-model or whole-attention-block savings.

Common misconception

“If GQA cuts KV-cache memory by 4x, the whole model becomes 4x smaller.” Table 7 shows why not: the complete attention block’s parameters (which include the unchanged Q and O projections) shrink by only 1.6x here, and the feed-forward sublayers (chapters 1-2) are untouched by H_{kv} entirely.

5.7 Compare and contrast

Table 8: MHA vs. GQA vs. MQA, compared without ranking one as strictly best – each row is a tradeoff, not a verdict

	MHA	GQA	MQA
H_{kv}	$H_{kv} = H_q$	$1 < H_{kv} < H_q$	$H_{kv} = 1$
K/V sharing	none	groups of H_q/H_{kv} query heads	all query heads
Logical KV-cache size	largest (1x)	intermediate (config-dependent)	smallest ($1x/H_q$)
K/V projection params	largest	intermediate	smallest
Representation flexibility	most (every head has its own K/V)	reduced, tunable via group size	least
Implementation considerations	simplest; no grouping logic	requires choosing a group size that divides H_q	simplest sharing case; largest capacity reduction
Common motivation	small models, or when cache memory isn’t the bottleneck	a tunable memory/flexibility tradeoff point	maximum cache/bandwidth savings when acceptable

Common misconception

“MQA is universally superior because it uses the least cache.” Least cache is not the only deployment factor. MQA gives every query head the least representational flexibility of the three (all of them share one key/value head, with no group boundary at all), which is exactly the tradeoff [7] introduces GQA to soften – reducing H_{kv} buys memory at a flexibility cost, and where that cost is acceptable is model- and workload-dependent, not a fixed rule.

5.8 Check your understanding

Full solutions are in the answer key (Section 8):

1. **Explain.** What is the one structural difference between MHA, MQA, and GQA – and what stays identical across all three? _____
2. **Calculation.** For $H_q = 8$ query heads and $H_{kv} = 2$ key/value heads, what is the group size, and how many query heads share each key/value head?

3. **Calculation.** Write the shapes of Q and K (per Equation 11) for $B = 1$, $T_q = 1$, $S = 100$, $H_q = 16$, $H_{kv} = 4$, $d_{\text{head}} = 64$. _____
4. **Quick check.** Which two of P_Q , P_K , P_V , P_O change when H_{kv} changes, holding H_q and d_{head} fixed?

5. **Misconception check.** True or false, and why: “GQA reduces the number of query heads.”

6. **Misconception check.** True or false, and why: “if GQA cuts a model’s KV-cache memory by 4x, the whole model becomes 4x smaller.” _____
7. **Calculation.** For $L = 24$, $H_q = 16$ (all used as H_{kv} , i.e. MHA), $d_{\text{head}} = 128$, $S = 4096$, $B = 1$, bf16: compute the logical KV-cache size in bytes and MiB using Equation 13.

(Space intentionally left for written answers if you are using this as a workbook exercise, not only a reading.)

5.9 Applied exercise

A serving workload needs long contexts and high concurrency. Candidate model A uses MHA; candidate B uses GQA, both otherwise the same size and S . Using Equation 13, compare their per-sequence cache cost at your own chosen (long) S and (large) B , then explicitly name at least one deployment factor besides cache memory (e.g. representation flexibility, implementation maturity, quality at the model’s specific task) that could still favor the MHA candidate despite its larger cache. A serving decision is not a cache-size comparison alone.

5.10 Interview lens

“Walk me through why GQA reduces KV-cache cost, and tell me what it does not automatically make cheaper.”
A strong answer names H_{kv} (not H_q) as the changing quantity, states Equation 13’s linear dependence on H_{kv} , and explicitly names what does *not* shrink proportionally: the query and output projections (P_Q , P_O), the number of query-side attention computations, and (per Table 7) the complete attention block’s total parameter count. An answer that only says “it uses less memory” without naming what stays fixed is incomplete for a systems-flavored interview.

5.11 Chapter recap

- H_q and H_{kv} are independent: MHA sets $H_{kv} = H_q$, MQA sets $H_{kv} = 1$, GQA sets $1 < H_{kv} < H_q$ – the same head-sharing axis at three points, never fewer query heads.
- Only the K/V projections’ shape and the KV-cache size depend on H_{kv} ; Q ’s shape, the output projection, and the number of query-side attention computations depend on H_q alone.
- The KV-cache formula ($2 \times B \times S \times L \times H_{kv} \times d_{\text{head}} \times \text{bytes_per_elem}$) is a logical minimum, not a measured GPU-memory number.
- K/V parameter and cache savings do not translate one-for-one into whole-model or whole-attention-block savings, and reducing H_{kv} does not proportionally cut attention FLOPs.
- None of MHA, GQA, or MQA is universally best – each trades cache memory against representational flexibility differently.

5.12 Sources and further reading

Primary sources for this chapter: [1] for the MHA baseline, [6] for MQA, [7] for GQA (including its up-training recipe for converting an existing MHA checkpoint, reported at roughly 5% of original pretraining compute – their own reported figure, not a universal constant). Secondary/supporting: [2] for cross-model framing, [8] for the KV-cache memory formula’s systems framing. A further latent-compression technique that shrinks the cache by a different mechanism than head-sharing exists and is left for a later chapter’s case studies, not covered here.

6 Answer Key: Chapter 1

Full solutions to Chapter 1’s “Check your understanding” questions (Section 3).

1. Quick check. What does the residual stream carry from one block to the next?

Answer. The running sum of the original embedding plus every sublayer update added so far – not just the most recent sublayer’s raw output. Per the residual update $x^{(l+1)} = x^{(l)} + f(\text{Norm}(x^{(l)}))$, each block adds its sublayer’s output onto the existing stream value; it never replaces the stream with just the new sublayer’s output.

Common trap: “the output of the most recent attention or MLP sublayer” – that describes only the update being added in that step, not the accumulated stream value.

2. Explain. Why does a decoder-only objective allow one-token- at-a-time generation?

Answer. Because the causal mask restricts every position to attending only to itself and earlier positions, the same forward-pass structure used in training also directly defines how to generate: at each position, produce a next-token distribution conditioned only on what came before. A bidirectional encoder’s positions can attend to *later* positions too, so its representation at a given position already depends on tokens that would not exist yet during generation – that structure cannot be reused one token at a time the way a causal one can.

Common trap: “decoder-only models are just smaller/simpler” – size is unrelated; the difference is specifically about which positions are allowed to attend to which.

3. Calculation. $d_{\text{model}} = 8$, $H_q = 2$, $S = 5$: find d_{head} and the shape of $QK^\top$ for one head.

Answer. $d_{\text{head}} = d_{\text{model}}/H_q = 8/2 = 4$. The raw attention score matrix for one head is $(S, S) = (5, 5)$ – one score per query/key pair, regardless of d_{head} .

Common trap: $(S, d_{\text{head}}) = (5, 4)$ – this confuses the per-head attention *output* shape (which is (S, d_{head})) with the score matrix shape (always (S, S)).

4. Misconception check. “Causal masking means each token can only attend to the immediately previous token.”

Answer. False. Causal masking allows a position to attend to *every* position at or before it ($j \leq i$), not only the single immediately preceding position. The mask is 0 (allowed) for every $j \leq i$ and $-\infty$ (blocked) only for $j > i$ – a full lower-triangular region, not a single diagonal band.

Why: conflating causal masking with a fixed local window (sliding-window attention, a separate mechanism covered in a later chapter). Causal masking alone imposes no limit on how far back a position can look.

5. Misconception check. “The residual stream is a separate memory database the model writes facts into and reads back later.”

Answer. False. The residual stream is a single vector per position, updated additively in place – not a separate, queryable store. Every sublayer’s output is added directly onto the same fixed-width vector that already exists at that position; there is no separate storage structure and no mechanism for a later block to selectively “query” what an earlier block specifically wrote. It can only read the accumulated sum.

Why: taking this chapter’s own receipt analogy too literally – a real receipt’s lines stay individually readable; the residual stream’s contributions are summed into one vector and cannot be individually read back out. This is exactly the analogy’s stated breakdown point.

6. Design exercise (applied exercise). Full tensor-shape trace for your own toy config.

Answer. For config $(d_{\text{model}}, L, H_q)$ with d_{model} divisible by H_q : input vector $(d_{\text{model}},)$; $Q/K/V$ before splitting, $(d_{\text{model}},)$ each (or (S, d_{model}) for a full sequence); after splitting into heads, (H_q, d_{head}) where $d_{\text{head}} = d_{\text{model}}/H_q$; per-head attention output, $(d_{\text{head}},)$ per head; concatenated across heads, back to $(d_{\text{model}},)$; final vector added to the residual stream, $(d_{\text{model}},)$ – matching the input shape exactly, since addition requires matching shapes.

Common trap: forgetting to concatenate the per-head outputs back to d_{model} width before the output projection and residual addition – leaving the answer at the per-head shape.

7. Interview practice. “Walk me through what happens differently between processing a 500-token prompt versus generating the 501st token.”

Answer. A strong answer: (1) names prefill – conceptually, all 500 positions evaluated together, building per-layer K/V state for every position (a serving engine may chunk this for scheduling reasons without changing the logical picture); (2) names the decode step – exactly one new position processed, computing only its own new K/V and reusing the 500 cached K/V rows unchanged; (3) explicitly states that generating the 501st token does not reprocess or re-read the first 500 tokens.

Interview criteria: does the answer name both phases correctly, does it state what specifically is reused (K/V rows) versus recomputed (the one new position’s K/V), and does it avoid the “re-reads the whole prompt” framing this chapter’s headline misconception warns against?

Common trap: “decoding is faster because the prompt is already processed,” without stating what specifically is reused or why that reuse is what makes it faster.

7 Answer Key: Chapter 2

Full solutions to Chapter 2’s “Check your understanding” questions (Section 4).

1. Calculation. RoPE, positions (1,0) and (3,2), both relative offset 1: what’s guaranteed, what isn’t?

Answer. RoPE guarantees the *angle* between the rotated query and key vectors is identical in both cases, because each vector is rotated by an amount tied to its own position, so only the relative offset (1, in both cases) survives. It does **not** guarantee the dot products are numerically equal in general – the dot product also depends on the original (pre-rotation) vectors’ magnitudes and directions, which can differ between the two pairs.

Common trap: claiming RoPE guarantees identical dot products for any two pairs with the same relative offset – this conflates the angle (guaranteed to match) with the dot product (which also depends on vector magnitudes).

2. Compare. Two operations RMSNorm omits relative to LayerNorm.

Answer. Mean-centering (subtracting the vector’s mean before rescaling) and the learned bias term (an additive per-dimension shift after rescaling). LayerNorm computes $(x - \text{mean}(x))/\text{std}(x) \odot \gamma + \beta$;

RMSNorm computes $x/\text{RMS}_\epsilon(x) \odot \gamma$ – no mean subtraction and no β term (the ϵ inside RMS_ϵ is a fixed numerical-stability constant, not a learned bias).

Common trap: naming only the missing bias and missing that mean-centering is also gone – dropping mean-centering is the more consequential of the two omissions.

3. Explain. Why three matrices for a gated feed-forward layer?

Answer. The third matrix (W_3 , the “value” branch) provides a second, purely linear projection of the input that gets elementwise-multiplied against the first matrix’s activated output (W_1 , the “gate” branch) before the shared output projection (W_2). W_1 alone only produces one activated signal; W_3 gives something for that signal to gate.

Common trap: describing the third matrix as making the network “deeper” – it does not; W_1 and W_3 are parallel branches computed from the same input, not a longer sequential chain.

4. Calculation. Plain MLP, $d_{\text{model}} = 16$, $d_{\text{ff}} = 64$: parameter count.

Answer. $2 d_{\text{model}} d_{\text{ff}} = 2 \times 16 \times 64 = 2048$ parameters, from this chapter’s plain-MLP formula instantiated at these dimensions and checked programmatically (see the Build and Version Note).

Common trap: computing only one matrix’s parameter count (1024) and forgetting the second, or applying the gated-MLP (3x) formula by mistake.

5. Misconception check. Pre-norm’s popularity: stability or quality?

Answer. Mainly training stability at depth, not final model quality. Pre-norm trades away some representational capacity for a more stable gradient path through an un-normalized residual stream as L grows. Sandwich-norm’s existence (paying for a *second* normalization step around the same sublayer) is direct evidence that pre-norm alone is understood to give something up, not that it is a strict improvement.

Common trap: treating pre-norm as simply “the new correct default” with no acknowledged tradeoff.

6. Misconception check. “RoPE adds a learned position vector to every token embedding.”

Answer. False. RoPE rotates the query and key vectors *inside* the attention computation, at every layer that has attention – it adds nothing to the embedding table, and the rotation angles are fixed by position, not learned.

Why: conflating RoPE with learned absolute positional embeddings (an older, different scheme that *does* add a learned per-position vector directly to the embedding).

7. Misconception check. “SwiGLU makes the feed-forward block sparse like an MoE layer.”

Answer. False. Every weight in a SwiGLU feed-forward layer is used for every token, exactly like a plain feed-forward layer. Sparsity, in the mixture-of-experts sense, means a *router* activates only a subset of available parameters per token – SwiGLU has no router and always uses 100% of its own weights.

Why: the word “gated” sounds like it could mean “selecting a subset,” but the gate here is a continuous, dense elementwise multiply applied to every token, not a discrete routing decision.

8. Design exercise (applied exercise). Reading a model config excerpt.

Answer. Given `hidden_size: 2048, num_attention_heads: 16, intermediate_size: 5504, hidden_act: "silu", rms_norm_eps: 1e-5, tie_word_embeddings: false, rope_theta: 10000.0:`

- **Normalization:** RMSNorm – the `rms_norm_eps` field name is the giveaway.
- **Gated feed-forward:** yes – `hidden_act: "silu"` matches SwiGLU’s Swish/SiLU convention, and `rope_theta` separately confirms RoPE is in use.
- **Ratio check:** $5504/2048 \approx 2.69$, close to $\frac{2}{3} \times 4 \approx 2.67$ – a *gated-compensated* ratio, not the plain 4x ratio.
- **Embeddings:** untied (`tie_word_embeddings: false` is explicit).

Common trap: assuming any roughly-4x `intermediate_size` implies a plain (ungated) MLP – the compensated gated ratio is close enough to other plausible ratios that the activation-function field is a more reliable signal than the ratio alone.

9. Interview practice. “Why RoPE instead of learned positional embeddings?”

Answer. A strong answer states RoPE’s relative-position mechanism as the reason, and names at least one concrete consequence: no learned positional parameters at all, and generally better behavior beyond the training length than a learned table with a fixed maximum.

Interview criteria: does the answer name the actual mechanism rather than asserting RoPE is “more modern”; does it name a specific, checkable consequence rather than a vague quality claim?

10. Interview practice. “Walk me through what SwiGLU changes about the MLP?”

Answer. A strong answer describes the extra parallel branch and the elementwise multiply before the shared output projection, and explicitly states the parameter tradeoff: at matched expansion width, three matrices cost 1.5x what two matrices cost, which is why real models often shrink the expansion width to compensate.

Interview criteria: does the answer proactively raise the parameter-cost tradeoff, rather than needing to be prompted for it – a systems-flavored interviewer will very likely ask a follow-up about cost if the candidate doesn’t raise it first.

8 Answer Key: Chapter 3

Full solutions to Chapter 3’s “Check your understanding” questions (Section 5).

1. Explain. What is the one structural difference between MHA, MQA, and GQA – and what stays identical across all three?

Answer. The one difference is H_{kv} : MHA sets $H_{kv} = H_q$ (every query head has its own key/value head), MQA sets $H_{kv} = 1$ (all query heads share one), and GQA sets $1 < H_{kv} < H_q$ (groups of query heads each share one). H_q itself, the query projection’s shape, the output projection’s shape, and the number of query-side attention computations are identical across all three.

Common trap: describing MHA/MQA/GQA as three unrelated mechanisms rather than three points on the same H_{kv} axis.

2. Calculation. For $H_q = 8$ query heads and $H_{kv} = 2$ key/value heads, what is the group size, and how many query heads share each key/value head?

Answer. Group size $= H_q / H_{kv} = 8 / 2 = 4$. Each key/value head is shared by 4 query heads.

Common trap: dividing H_{kv} by H_q instead of H_q by H_{kv} , which gives a fraction instead of a head count.

3. Calculation. Write the shapes of Q and K for $B = 1, T_q = 1, S = 100, H_q = 16, H_{kv} = 4, d_{\text{head}} = 64$.

Answer. $Q : (1, 1, 16, 64)$. $K : (1, 100, 4, 64)$.

Common trap: using S in place of T_q for Q 's second axis (or vice versa for K) – Q 's position axis is how many *new* positions this step computes queries for, while K/V 's position axis is how many positions are *retained* in the cache.

4. Quick check. Which two of P_Q, P_K, P_V, P_O change when H_{kv} changes, holding H_q and d_{head} fixed?

Answer. P_K and P_V . $P_Q = d_{\text{model}} H_q d_{\text{head}}$ and $P_O = H_q d_{\text{head}} d_{\text{model}}$ depend only on H_q (and $d_{\text{head}}/d_{\text{model}}$), not H_{kv} .

Common trap: assuming all four projections shrink together, since they are all part of “the attention block.”

5. Misconception check. “GQA reduces the number of query heads.”

Answer. False. H_q is unchanged by GQA (or MQA) – only H_{kv} changes; Figure 9 draws the same number of query heads in every panel. This misconception usually comes from conflating “fewer heads are involved in some sense” with “fewer query heads specifically,” when what actually shrinks is the key/value side only.

6. Misconception check. “If GQA cuts a model’s KV-cache memory by 4x, the whole model becomes 4x smaller.”

Answer. False. Table 7 shows the complete attention block (including the unchanged P_Q and P_O) shrinks by only 1.6x here, not 4x, and the feed-forward sublayers are untouched by H_{kv} entirely. The cache formula and the model’s total parameter count share one input (H_{kv}) but respond to it very differently – the cache formula has no $Q/O/MLP$ terms to hold anything back, while the parameter count does.

7. Calculation. For $L = 24, H_q = 16$ (MHA, so $H_{kv} = H_q = 16$), $d_{\text{head}} = 128, S = 4096, B = 1, \text{bf16}$: compute the logical KV-cache size in bytes and MiB.

Answer. $M_{KV} = 2 \times 1 \times 4096 \times 24 \times 16 \times 128 \times 2 = 805,306,368$ bytes = 768 MiB exactly (verified programmatically; see the Build and Version Note).

Common trap: forgetting the factor of 2 for keys and values, which would understate the answer by exactly half (384 MiB).

8. Design exercise (applied exercise). Compare a candidate MHA model and a candidate GQA model for a long-context, high-concurrency serving workload.

Answer. At the same base size and S , the GQA candidate’s per-sequence cache is smaller by the ratio $H_{kv, \text{GQA}}/H_q$ (per Equation 13), which at high concurrency directly increases how many sequences fit in a fixed memory budget – usually the dominant argument for GQA here. But the MHA candidate could still win on task-specific quality (if trained, not just uptrained, with full per-head capacity) or serving-stack maturity. A complete answer performs the ratio comparison *and* names a concrete non-cache factor, not a vague “it depends.”

9. Interview practice. “Walk me through why GQA reduces KV-cache cost, and tell me what it does not automatically make cheaper.”

Answer. A strong answer: (1) names H_{kv} , not H_q , as the quantity GQA changes; (2) states Equation 13’s linear dependence on H_{kv} as the mechanism; (3) proactively names what does *not* shrink proportionally – P_Q/P_O , the number of query-side attention computations, and the complete attention block’s total parameter count (per Table 7).

Interview criteria: does the answer name the specific mechanism rather than a vague “less memory” claim, and proactively name at least one thing that stays fixed size rather than needing to be prompted for it? Stopping at “GQA uses less memory” is the common trap here – a systems-flavored interviewer will very likely ask exactly that follow-up.

9 Build and Version Note

- **Generated:** 2026-09-23
- **Git commit:** 6f14543f039409f8ba9edcfb4a11ffbb5cle5d5f (pre-commit -- working tree had uncommitted changes at build time)
- **Source-registry verification date referenced by this build:** src-19, src-20, src-21, src-22, src-23, src-24 (primary sources for Ch.1-3) last verified 2026-09-22; see sources/registry.yaml
- **Pilot status:** Three-chapter pilot (Chapters 1-3 of 8) – Chapters 4-8 are planned but not drafted; Chapter 3 defers multi-head latent attention (MLA), see outline.yaml’s ch3.drafting_note

9.1 Provenance

- Tied-embeddings usage (Chapter 2) has no dedicated primary source; the sandwich-norm claim (Chapter 2) leans on a secondary source rather than a model’s own technical report. Both are tracked sourcing gaps, not unsupported claims.
- All worked-example numbers in Chapters 1-3 (attention weights, parameter counts, KV-cache sizes) are computed by version-controlled project scripts and checked by the project’s automated test suite, not hand-derived.
- Figures are generated from version-controlled scripts using a fixed visual-style configuration, not drawn freehand; each figure’s caption credits its scholarly/technical source where one applies.
- Chapter 3’s KV-cache and attention-projection-parameter figures/formulas are LOGICAL, theoretical minimums, not measured GPU memory – they exclude allocator overhead, page metadata, fragmentation, and framework buffers.

This note is generated by `scripts/generate_build_note.py` at build time – do not hand-edit `includes/build-version-note.qmd`; regenerate it instead.

Bibliography

- [1] A. Vaswani *et al.*, “Attention Is All You Need.” [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [2] S. Raschka, “The Big LLM Architecture Comparison.” [Online]. Available: <https://magazine.sebastianraschka.com/p/the-big-llm-architecture-comparison>
- [3] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” [Online]. Available: <https://arxiv.org/abs/2104.09864>
- [4] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization.” [Online]. Available: <https://arxiv.org/abs/1910.07467>

- [5] N. Shazeer, “GLU Variants Improve Transformer.” [Online]. Available: <https://arxiv.org/abs/2002.05202>
- [6] N. Shazeer, “Fast Transformer Decoding: One Write-Head is All You Need.” [Online]. Available: <https://arxiv.org/abs/1911.02150>
- [7] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.” [Online]. Available: <https://arxiv.org/abs/2305.13245>
- [8] JAX-ML / Google, “How To Scale Your Model.” [Online]. Available: <https://jax-ml.github.io/scaling-book/>